

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG  
KHOA AN TOÀN THÔNG TIN**



**BÁO CÁO BÀI TẬP LỚN  
HỌC PHẦN: AN TOÀN MẠNG NÂNG CAO  
MÃ HỌC PHẦN: INT1483**

**ĐỀ TÀI: TÌM HIỂU VỀ GIAO THỨC BẢO MẬT SSL/TLS**

Các sinh viên thực hiện (trưởng nhóm xếp số 1):

|             |                  |
|-------------|------------------|
| B22DCATT172 | VŨ THỊ MỸ LỆ     |
| B22DCATT144 | NGUYỄN QUANG HUY |
| B22DCATT148 | PHẠM ĐĂNG HUY    |
| B22DCATT156 | HOÀNG VĂN HƯỚNG  |
| B22DCATT164 | HOÀNG ANH KHOA   |
| B22DCATT168 | NGUYỄN HOÀNG LÂM |

Tên nhóm: 6

Tên lớp: D22AT-02

Giảng viên hướng dẫn: PGS.TS. Hoàng Xuân Dâu

**HÀ NỘI 3-2026**

## PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

| TT | Công việc / Nhiệm vụ                                                                                                                              | SV thực hiện     | Thời hạn hoàn thành |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------|------------------|---------------------|
| 1  | Phân công công việc, lên kịch bản báo cáo; Tìm hiểu về TLS Handshake và cơ chế hoạt động của TLS; Chỉnh sửa nội dung; Tổng hợp báo cáo; Làm slide | Vũ Thị Mỹ Lệ     | 22/02/2025          |
| 2  | Tìm hiểu về tấn công hạ cấp TLS; Làm slide                                                                                                        | Nguyễn Quang Huy | 22/02/2025          |
| 3  | Tìm hiểu về tấn công vào điểm yếu toán học TLS; Làm slide                                                                                         | Phạm Đăng Huy    | 22/02/2025          |
| 4  | Tìm hiểu tổng quan và ứng dụng của SSL/TLS; Làm Slide                                                                                             | Hoàng Văn Hường  | 22/02/2025          |
| 5  | Tìm hiểu về TLS Record Protocol, Alert Protocol, ChangeCipherSpec Protocol; Làm Slide                                                             | Hoàng Anh Khoa   | 22/02/2025          |
| 6  | Thực hiện demo; Làm Slide                                                                                                                         | Nguyễn Hoàng Lâm | 22/02/2025          |

## NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

| TT | SV thực hiện     | Thái độ tham gia | Mức hoàn thành CV | Kỹ năng giao tiếp | Kỹ năng hợp tác | Kỹ năng lãnh đạo |
|----|------------------|------------------|-------------------|-------------------|-----------------|------------------|
| 1  | Vũ Thị Mỹ Lệ     | 5                | 5                 | 5                 | 4               | 4                |
| 2  | Nguyễn Quang Huy | 4                | 4                 | 3                 | 3               | 0                |
| 3  | Phạm Đăng Huy    | 3                | 3                 | 3                 | 3               | 0                |
| 4  | Hoàng Văn Hường  | 5                | 5                 | 4                 | 3               | 2                |
| 5  | Hoàng Anh Khoa   | 4                | 4                 | 2                 | 2               | 0                |
| 6  | Nguyễn Hoàng Lâm | 4                | 3                 | 3                 | 3               | 0                |

### ***Ghi chú:***

- Thái độ tham gia: Đánh giá điểm thái độ tham gia công việc chung của nhóm (từ 0: không tham gia, đến 5: chủ động, tích cực).

- Mức hoàn thành CV: Đánh giá điểm mức độ hoàn thành công việc được giao (từ 0: không hoàn thành, đến 5: hoàn thành xuất sắc).
- Kỹ năng giao tiếp: Đánh giá điểm khả năng tương tác, giao tiếp trong nhóm (từ 0: không hoặc giao tiếp rất yếu, đến 5: giao tiếp xuất sắc).
- Kỹ năng hợp tác: Đánh giá điểm khả năng hợp tác, hỗ trợ lẫn nhau, giải quyết mâu thuẫn, xung đột
- Kỹ năng lãnh đạo: Đánh giá điểm khả năng lãnh đạo (từ 0: không có khả năng lãnh đạo, đến 5: có khả năng lãnh đạo tốt, tổ chức và điều phối công việc trong nhóm hiệu quả).

# MỤC LỤC

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| MỤC LỤC .....                                                       | 4         |
| DANH MỤC CÁC HÌNH VẼ .....                                          | 6         |
| DANH MỤC CÁC BẢNG BIỂU .....                                        | 6         |
| DANH MỤC CÁC TỪ VIẾT TẮT.....                                       | 7         |
| MỞ ĐẦU .....                                                        | 8         |
| <b>CHƯƠNG 1. TỔNG QUAN VỀ GIAO THỨC BẢO MẬT SSL/TLS .....</b>       | <b>9</b>  |
| 1.1 Khái niệm về SSL/TLS .....                                      | 9         |
| 1.2 Vai trò của giao thức SSL/TLS .....                             | 9         |
| 1.3 Vị trí của SSL/TLS trong mô hình mạng .....                     | 9         |
| 1.4 Các phiên bản của SSL/TLS .....                                 | 10        |
| <b>CHƯƠNG 2. KIẾN TRÚC CỦA GIAO THỨC SSL/TLS .....</b>              | <b>11</b> |
| 2.1 Kiến trúc của giao thức SSL/TLS.....                            | 11        |
| 2.2 SSL/TLS Record Protocol.....                                    | 11        |
| 2.3 SSL/TLS Alert Protocol.....                                     | 14        |
| 2.4 SSL/TLS Change Cipher Spec Protocol .....                       | 14        |
| 2.5 SSL/TLS Handshake Protocol.....                                 | 15        |
| <b>CHƯƠNG 3. CƠ CHẾ HOẠT ĐỘNG CỦA GIAO THỨC SSL/TLS .....</b>       | <b>21</b> |
| 3.1 Tổng quan.....                                                  | 21        |
| 3.2 Giai đoạn Đàm phán sơ bộ và xác thực .....                      | 21        |
| 3.3 Giai đoạn Dẫn xuất Khóa .....                                   | 21        |
| 3.4 Giai đoạn Kích hoạt Bảo mật .....                               | 22        |
| 3.5 Giai đoạn Truyền dữ liệu .....                                  | 22        |
| <b>CHƯƠNG 4. CÁC LỖ HỔNG VÀ CÁC DẠNG TẤN CÔNG VÀO SSL/TLS .....</b> | <b>23</b> |
| 4.1 Mở đầu .....                                                    | 23        |
| 4.2 Tấn công hạ cấp (Downgrade Attacks).....                        | 23        |
| 4.3 Tấn công vào điểm yếu toán học .....                            | 25        |
| <b>CHƯƠNG 5. ỨNG DỤNG CỦA GIAO THỨC SSL/TLS .....</b>               | <b>29</b> |
| 5.1 Mở đầu .....                                                    | 29        |
| 5.2 Ứng dụng trong bảo mật Web (HTTPS) .....                        | 29        |
| 5.3 Ứng dụng trong bảo mật thư điện tử.....                         | 30        |
| 5.4 Ứng dụng trong VPN và truy cập từ xa .....                      | 30        |
| 5.5 Ứng dụng trong các dịch vụ thư mục và xác thực.....             | 30        |
| 5.6 Ứng dụng trong API và kiến trúc Microservices .....             | 31        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| 5.7 Ứng dụng trong điện toán đám mây và container ..... | 31        |
| 5.8 Ứng dụng trong bảo mật truyền file .....            | 31        |
| <b>CHƯƠNG 6. THỰC NGHIỆM TRIỂN KHAI SSL/TLS .....</b>   | <b>32</b> |
| 6.1 Các bước cài đặt .....                              | 32        |
| 6.2 Kết quả thu được.....                               | 34        |
| KẾT LUẬN.....                                           | 36        |
| TÀI LIỆU THAM KHẢO .....                                | 37        |

## DANH MỤC CÁC HÌNH VẼ

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| <i>Hình 1</i> Vị trí của TLS trong mô hình OSI .....                                      | 9  |
| <i>Hình 2</i> Các phiên bản của SSL/TLS.....                                              | 10 |
| <i>Hình 3</i> Kiến trúc của SSL/TLS .....                                                 | 11 |
| <i>Hình 4</i> Quy trình xử lý dữ liệu của TLS Record Protocol.....                        | 12 |
| <i>Hình 5</i> Quy trình Full Handshake .....                                              | 16 |
| <i>Hình 6</i> Cơ chế khôi phục phiên.....                                                 | 20 |
| <i>Hình 7</i> Cơ chế hoạt động của SSL/TLS .....                                          | 21 |
| <i>Hình 8</i> Kẻ tấn công liên tục phá vỡ kết nối đến khi máy khách hạ xuống SSL 3.0..... | 25 |
| <i>Hình 9</i> Tấn công SWEET32 .....                                                      | 27 |
| <i>Hình 10</i> HTTP và HTTPS .....                                                        | 29 |
| <i>Hình 11</i> Sơ đồ cấu hình SMTP với STARTTLS qua BIG-IP.....                           | 30 |

## DANH MỤC CÁC BẢNG BIỂU

## DANH MỤC CÁC TỪ VIẾT TẮT

| <b>Từ viết tắt</b> | <b>Thuật ngữ tiếng Anh/Giải thích</b>  | <b>Thuật ngữ tiếng Việt/Giải thích</b>  |
|--------------------|----------------------------------------|-----------------------------------------|
| SSL                | Secure Sockets Layer                   | Giao thức bảo mật lớp công bảo mật      |
| TLS                | Transport Layer Security               | Giao thức bảo mật tầng giao vận         |
| HTTPS              | HyperText Transfer Protocol Secure     | Giao thức truyền siêu văn bản bảo mật   |
| HTTP               | HyperText Transfer Protocol            | Giao thức truyền siêu văn bản           |
| TCP                | Transmission Control Protocol          | Giao thức điều khiển truyền tải         |
| CA                 | Certificate Authority                  | Tổ chức cấp chứng chỉ số                |
| PKI                | Public Key Infrastructure              | Hạ tầng khóa công khai                  |
| HMAC               | Hash-based Message Authentication Code | Mã xác thực thông điệp dựa trên hàm băm |
| MAC                | Message Authentication Code            | Mã xác thực thông điệp                  |
| PRF                | Pseudorandom Function                  | Hàm giả ngẫu nhiên                      |
| VPN                | Virtual Private Network                | Mạng riêng ảo                           |
| SMTP               | Simple Mail Transfer Protocol          | Giao thức gửi thư điện tử               |
| IMAP               | Internet Message Access Protocol       | Giao thức truy cập thư điện tử          |
| POP3               | Post Office Protocol version 3         | Giao thức nhận thư điện tử              |

## MỞ ĐẦU

Trong bối cảnh hạ tầng số và các dịch vụ trực tuyến phát triển mạnh mẽ, vấn đề đảm bảo an toàn thông tin trong quá trình truyền dữ liệu qua mạng Internet ngày càng trở nên cấp thiết. Các nguy cơ như nghe lén, giả mạo, tấn công trung gian hay thay đổi nội dung truyền tải có thể gây ra những hậu quả nghiêm trọng đối với cá nhân và tổ chức. Do đó, việc xây dựng và triển khai các cơ chế bảo mật ở tầng truyền thông là yêu cầu bắt buộc trong các hệ thống thông tin hiện đại.

SSL (Secure Sockets Layer) và TLS (Transport Layer Security) là các giao thức bảo mật được thiết kế nhằm đảm bảo ba mục tiêu cốt lõi của an toàn thông tin: tính bảo mật (Confidentiality), tính toàn vẹn (Integrity) và tính xác thực (Authentication). Trong đó, TLS – phiên bản kế nhiệm của SSL – hiện đang là tiêu chuẩn bảo mật được sử dụng rộng rãi trong nhiều dịch vụ. Việc hiểu rõ kiến trúc, cơ chế hoạt động cũng như các lỗ hổng tiềm ẩn của SSL/TLS có ý nghĩa quan trọng trong quá trình thiết kế, vận hành và bảo vệ hệ thống mạng.

Báo cáo này được xây dựng nhằm nghiên cứu toàn diện về giao thức SSL/TLS từ lý thuyết đến thực nghiệm. Nội dung báo cáo được tổ chức thành các chương như sau:

- **Chương 1: Tổng quan về giao thức SSL/TLS**  
Trình bày các khái niệm cơ bản, vai trò của SSL/TLS trong đảm bảo an toàn thông tin, vị trí của giao thức trong mô hình mạng, cùng với quá trình phát triển và các phiên bản của SSL/TLS.
- **Chương 2: Kiến trúc của giao thức SSL/TLS**  
Trình bày cấu trúc phân lớp của SSL/TLS, các giao thức thành phần như Record Protocol, Handshake Protocol, ChangeCipherSpec Protocol và Alert Protocol.
- **Chương 3: Cơ chế hoạt động của giao thức SSL/TLS**  
Phân tích chi tiết quá trình bắt tay (Handshake), cơ chế dẫn xuất khóa, kích hoạt bảo mật, truyền dữ liệu và cơ chế khôi phục phiên làm việc.
- **Chương 4: Các lỗ hổng và dạng tấn công vào SSL/TLS**  
Trình bày các nhóm lỗ hổng phổ biến, phân tích nguyên nhân, cách thức tấn công, hậu quả và biện pháp phòng chống nhằm tăng cường an toàn cho hệ thống.
- **Chương 5: Ứng dụng của giao thức SSL/TLS**  
Phân tích các ứng dụng thực tiễn của SSL/TLS trong bảo mật web (HTTPS), thư điện tử, VPN, dịch vụ thư mục, API, microservices, điện toán đám mây và truyền file.
- **Chương 6: Thực nghiệm triển khai SSL/TLS**  
Thực hiện cài đặt và cấu hình một website chạy trên giao thức HTTPS sử dụng chứng chỉ SSL tự ký, kiểm tra chứng chỉ và phân tích kết quả thu được.



# CHƯƠNG 1. TỔNG QUAN VỀ GIAO THỨC BẢO MẬT SSL/TLS

## 1.1 Khái niệm về SSL/TLS

SSL (Secure Sockets Layer) và TLS (Transport Layer Security) là các giao thức bảo mật được thiết kế nhằm đảm bảo an toàn cho dữ liệu truyền tải qua mạng máy tính, đặc biệt là mạng Internet. Hai giao thức này cho phép thiết lập một kênh truyền thông an toàn giữa hai thực thể (thường là client và server) bằng cách sử dụng các cơ chế mã hóa và xác thực.

SSL là giao thức được phát triển đầu tiên bởi Netscape vào những năm 1990. Tuy nhiên, do tồn tại nhiều lỗ hổng bảo mật nghiêm trọng, SSL đã dần bị loại bỏ. TLS ra đời như một phiên bản kế nhiệm và cải tiến của SSL, được chuẩn hóa bởi IETF và hiện là tiêu chuẩn bảo mật được sử dụng rộng rãi trong các hệ thống mạng hiện nay. Trên thực tế, thuật ngữ “SSL” vẫn thường được sử dụng, nhưng hầu hết các hệ thống hiện đại đều đang dùng TLS.

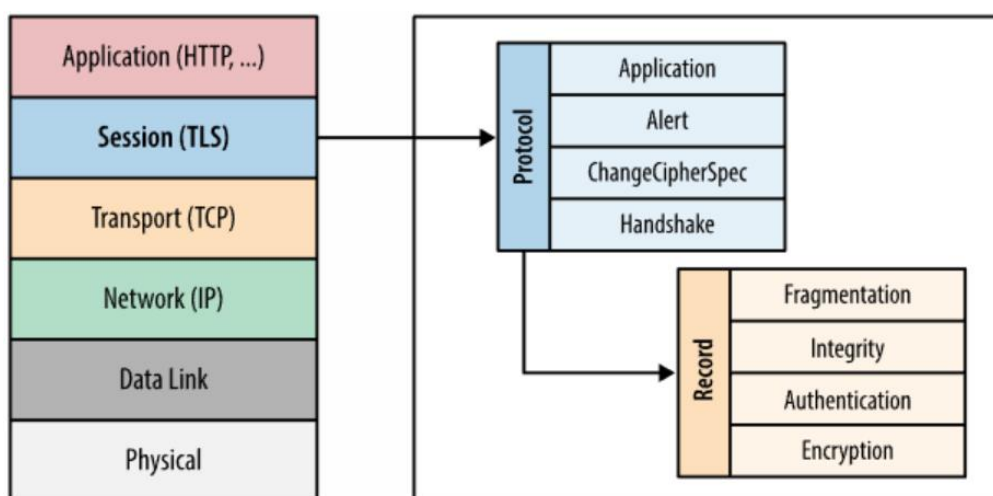
## 1.2 Vai trò của giao thức SSL/TLS

SSL/TLS đóng vai trò quan trọng trong việc bảo vệ thông tin khi truyền qua mạng, thể hiện qua ba mục tiêu chính của an toàn thông tin:

- **Bảo mật (Confidentiality):** Dữ liệu được mã hóa trong quá trình truyền, ngăn chặn việc nghe lén hoặc đánh cắp thông tin bởi các bên không được phép.
- **Toàn vẹn dữ liệu (Integrity):** SSL/TLS sử dụng các hàm băm và mã xác thực thông điệp để đảm bảo dữ liệu không bị thay đổi trong quá trình truyền tải.
- **Xác thực (Authentication):** Giao thức cho phép client xác thực danh tính của server thông qua chứng chỉ số, từ đó tránh nguy cơ kết nối đến các máy chủ giả mạo. Trong một số trường hợp, SSL/TLS còn hỗ trợ xác thực hai chiều (mutual authentication).

## 1.3 Vị trí của SSL/TLS trong mô hình mạng

SSL/TLS không được định nghĩa là một tầng riêng biệt trong mô hình OSI, mà hoạt động giữa tầng ứng dụng (Application Layer) và tầng giao vận (Transport Layer).

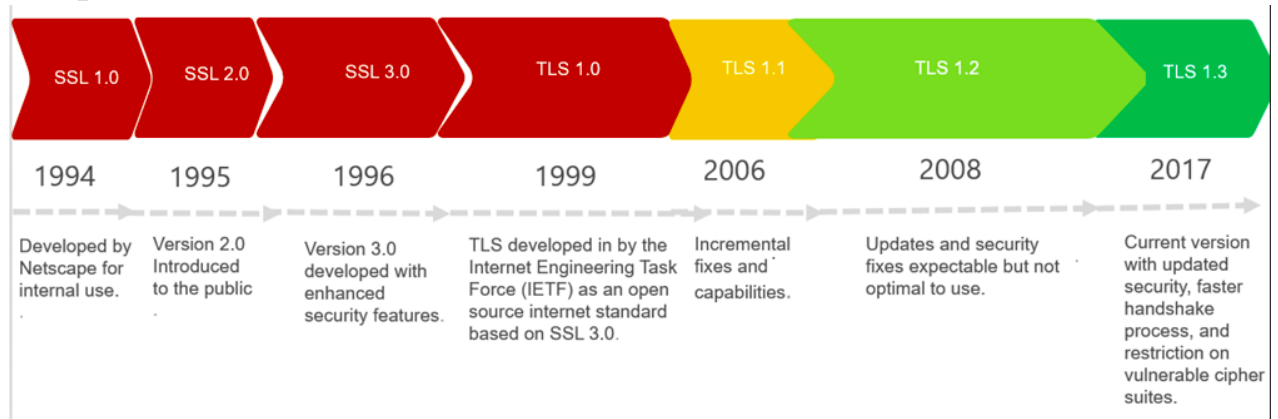


Hình 1 Vị trí của TLS trong mô hình OSI

Cụ thể, SSL/TLS nhận dữ liệu từ các giao thức tầng ứng dụng như HTTP, SMTP, FTP..., thực hiện mã hóa và xác thực, sau đó chuyển dữ liệu đã được bảo vệ xuống tầng

TCP để truyền đi. Cách tiếp cận này cho phép SSL/TLS hoạt động độc lập với ứng dụng, đồng thời có thể áp dụng cho nhiều giao thức khác nhau mà không cần thay đổi cấu trúc mạng nền tảng

## 1.4 Các phiên bản của SSL/TLS



Hình 2 Các phiên bản của SSL/TLS

### 1.4.1 Các phiên bản SSL

- SSL 1.0: Không được công bố chính thức do tồn tại nhiều lỗi thiết kế.
- SSL 2.0: Phiên bản đầu tiên được triển khai nhưng có nhiều điểm yếu nghiêm trọng, đã bị loại bỏ.
- SSL 3.0: Cải tiến so với SSL 2.0 nhưng vẫn tồn tại lỗ hổng bảo mật, điển hình là tấn công POODLE, hiện không còn được khuyến nghị sử dụng.

### 1.4.2 Các phiên bản TLS

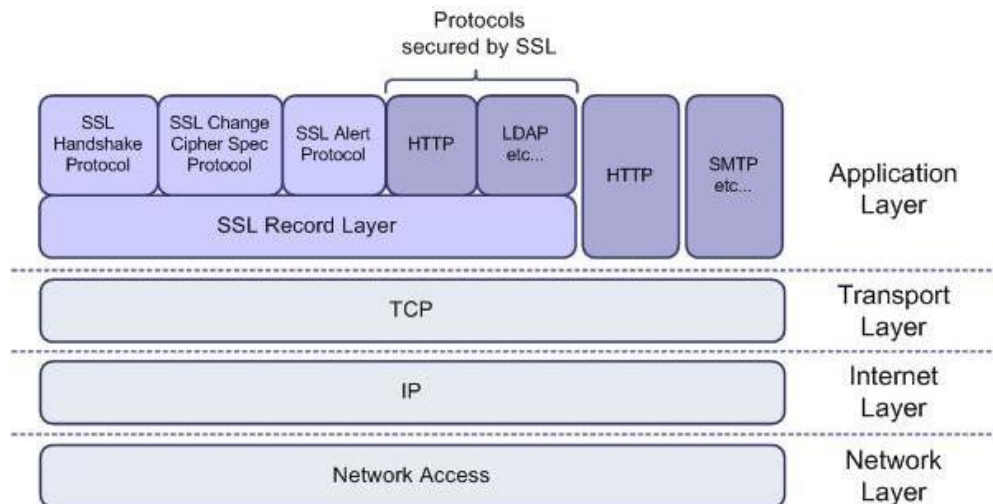
- TLS 1.0: Phiên bản đầu tiên kế thừa SSL 3.0, hiện đã không còn an toàn.
- TLS 1.1: Cải thiện một số vấn đề của TLS 1.0 nhưng vẫn bị coi là lỗi thời.
- TLS 1.2: Phiên bản được sử dụng phổ biến trong nhiều hệ thống hiện nay, hỗ trợ các thuật toán mã hóa mạnh và linh hoạt.
- TLS 1.3: Phiên bản mới nhất, tối ưu về hiệu năng và bảo mật, giảm số bước bắt tay (handshake), loại bỏ các thuật toán yếu và bắt buộc hỗ trợ tính năng bảo mật chuyển tiếp (Forward Secrecy).

Hiện nay, các tổ chức và hệ thống thông tin được khuyến nghị chỉ sử dụng TLS 1.2 hoặc TLS 1.3, đồng thời vô hiệu hóa hoàn toàn các phiên bản SSL và TLS cũ để đảm bảo an toàn thông tin.

## CHƯƠNG 2. KIẾN TRÚC CỦA GIAO THỨC SSL/TLS

### 2.1 Kiến trúc của giao thức SSL/TLS

Giao thức SSL/TLS không tồn tại như một thực thể đơn lẻ mà được cấu trúc dưới dạng một bộ giao thức phân lớp phức tạp. SSL/TLS được cấu thành từ bốn giao thức cốt lõi, hoạt động phối hợp chặt chẽ và nhịp nhàng với nhau để khởi tạo, duy trì cấu trúc, giám sát trạng thái và kết thúc một phiên giao dịch an toàn. Bốn giao thức này bao gồm: Giao thức Record (Record Protocol), Giao thức Handshake (Handshake Protocol), Giao thức ChangeCipherSpec (ChangeCipherSpec Protocol) và Giao thức Alert (Alert Protocol).



Hình 3 Kiến trúc của SSL/TLS

- SSL/TLS Handshake Protocol: Giao thức có nhiệm vụ trao đổi các thông điệp xác thực thực thể và thiết lập các thông số cho phiên làm việc.
- SSL/TLS Change Cipher Spec Protocol: Giao thức thiết lập việc sử dụng các bộ mã hóa được hỗ trợ bởi cả 2 bên tham gia truyền thông.
- SSL/TLS Alert Protocol: Giao thức cảnh báo.
- SSL/TLS Record Protocol: Giao thức có nhiệm vụ tạo đường hầm an toàn để chuyển thông tin đảm bảo tính bí mật, toàn vẹn và xác thực.

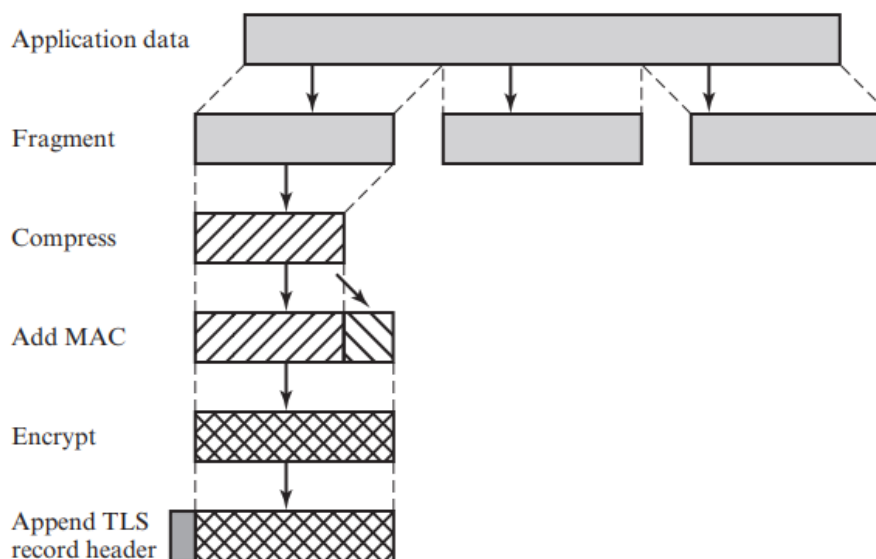
### 2.2 SSL/TLS Record Protocol

#### 2.2.1 Tổng quan về SSL/TLS Record Protocol

Giao thức Record đóng vai trò là nền tảng vận chuyển và là tầng thấp nhất trong hệ thống SSL/TLS. Nhiệm vụ của nó là cung cấp hai thuộc tính an ninh cốt lõi: tính bảo mật của dữ liệu (thông qua quá trình mã hóa) và tính toàn vẹn của thông điệp (thông qua các cơ chế xác thực). Mọi dữ liệu đi qua SSL/TLS, bao gồm cả dữ liệu của các giao thức ứng dụng cấp cao lẫn dữ liệu điều khiển của các giao thức Handshake hay Alert, đều phải được đóng gói và vận chuyển bởi giao thức Record.

#### 2.2.2 Quy trình xử lý dữ liệu của SSL/TLS Record Protocol

Quy trình xử lý dữ liệu của TLS Record Protocol được chia thành hai chiều: từ lúc nhận dữ liệu từ các giao thức cấp cao, xử lý bảo vệ, rồi đưa ra mạng; và chiều ngược lại khi nhận bản ghi từ mạng, kiểm tra, giải mã, sau đó chuyển lên đúng giao thức con.



Hình 4 Quy trình xử lý dữ liệu của TLS Record Protocol

#### 2.2.2.1 Chiều gửi

Khi các giao thức tầng trên (Handshake, Alert, ChangeCipherSpec, hoặc Application Data) muốn gửi dữ liệu, Record Protocol sẽ thực hiện lần lượt các bước sau:

##### 1. Phân mảnh (Fragmentation)

- Dữ liệu thô được nhận thành một luồng liên tục. Record Protocol sẽ băm/chia nhỏ luồng dữ liệu này thành các khối có kích thước dễ quản lý.
- Kích thước tối đa của mỗi khối phân mảnh không được vượt quá  $2^{14}$  bytes. Kết quả của bước này gọi là cấu trúc **TLSP Plaintext**.

##### 2. Nén (Compression - Tùy chọn)

- Khối dữ liệu phân mảnh sẽ được nén lại bằng thuật toán nén đã được định nghĩa trong "Trạng thái kết nối hiện tại" (Current Connection State).
- Lưu ý: Trong thực tế hiện nay (và đặc biệt là từ TLS 1.3), tính năng nén thường bị vô hiệu hóa (sử dụng thuật toán Null) để tránh các rủi ro bảo mật như tấn công CRIME. Kết quả bước này tạo ra **TLSC Compressed**.

##### 3. Tính toán MAC và Mã hóa (MAC & Encryption)

- Tại bước này, Record Protocol sử dụng các tham số bảo mật (khóa mã hóa, khóa MAC) từ "Trạng thái ghi hiện tại" (Current Write State).
- Thêm MAC: Một Mã xác thực thông điệp (MAC) được tính toán từ dữ liệu (cùng với số thứ tự ẩn - Sequence Number) và gắn vào sau dữ liệu để đảm bảo tính toàn vẹn.

- Mã hóa: Dữ liệu cùng với khối MAC (và có thể thêm phần padding - dữ liệu đệm nếu dùng mã hóa khối) sẽ được mã hóa bằng thuật toán mã hóa đối xứng (như AES).
- Trong các chuẩn hiện đại (như TLS 1.3 hoặc các Cipher Suite dùng AEAD như AES-GCM): Quá trình mã hóa và xác thực (MAC) được gộp chung vào một bước duy nhất rất an toàn.
- Kết quả của bước này tạo ra **TLSCiphertext**.

#### 4. Đóng gói (*Framing / Thêm Header*)

- Record Protocol gắn thêm một Header cố định dài 5 bytes vào trước khối dữ liệu đã mã hóa.
- Header này chứa:
  - Content Type (1 byte): Xác định loại dữ liệu chứa trong fragment. Các giá trị 20–23 tương ứng với bốn giao thức con. Đây là trường quan trọng nhất để phân luồng dữ liệu về đúng bộ xử lý.
  - Protocol Version (2 bytes): Trong TLS 1.3, trường này vẫn mang giá trị 0x0303 (tương ứng TLS 1.2) nhằm đảm bảo tương thích ngược; phiên bản thực sự được thương lượng thông qua extension supported versions trong ClientHello/ServerHello.
  - Length (2 bytes): Độ dài của fragment, không được vượt quá  $2^{14}$  byte. Nếu vượt quá, bên nhận phải kết thúc kết nối bằng alert record overflow.

#### 5. Truyền tải

- Bản ghi TLS hoàn chỉnh được đẩy xuống tầng Giao vận (TCP) để truyền qua mạng đến máy đích.

##### 2.2.2.2 Chiều nhận

Khi nhận được luồng byte từ TCP, Record Protocol sẽ thực hiện quy trình ngược lại một cách cực kỳ cẩn thận để tránh bị kẻ tấn công đánh lừa:

##### 1. Đọc Header và Tách bản ghi (*Record Assembly*)

- Hệ thống đọc 5 bytes đầu tiên để lấy thông tin Header.
- Dựa vào trường Length trong Header, Record Protocol biết được cần phải đọc thêm bao nhiêu bytes tiếp theo để có được một bản ghi TLSCiphertext trọn vẹn. (Nếu Length vượt quá giới hạn cho phép, kết nối sẽ bị ngắt).

##### 2. Giải mã và Xác thực (*Decryption & MAC Verification*)

- Sử dụng các tham số trong "Trạng thái đọc hiện tại" (Current Read State), Record Protocol tiến hành giải mã khối dữ liệu.
- Đồng thời, hệ thống tính toán lại giá trị MAC từ dữ liệu vừa giải mã (cùng với Sequence Number của bên nhận) và so sánh với MAC được gửi kèm.

### 3. Giải nén (Decompression)

- Nếu dữ liệu đã được nén ở chiều gửi (rất hiếm trong các hệ thống hiện đại), nó sẽ được giải nén để khôi phục lại văn bản rõ ban đầu (TLSPlaintext).

### 4. Ghép nối và Phân luồng

- Nếu dữ liệu của một thông điệp lớn bị phân mảnh thành nhiều bản ghi, chúng có thể được ghép nối lại.
- Dựa vào trường Content Type trong Header ban đầu, Record Protocol sẽ đẩy luồng dữ liệu sạch này lên đúng giao thức tầng trên tương ứng.

#### 2.2.3 Tính toán giá trị MAC/HMAC trong SSL/TLS

Công thức tổng quát:

$$\text{HMAC}_K(M) = H\left[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]\right]$$

- $H$ : Hàm băm mật mã được sử dụng (đối với các phiên bản TLS cũ, thường là MD5 hoặc SHA-1).
- $M$ : Thông điệp đầu vào (Message input) cần được xác thực.
- $K^+$ : Khóa bí mật được đệm thêm các bit 0 sao cho kết quả có độ dài bằng chính xác độ dài khối (block length) của hàm băm (Ví dụ: đối với MD5 và SHA-1, độ dài khối là 512 bits).
- $\text{ipad}$ : Lớp đệm trong (inner pad), bao gồm chuỗi nhị phân 00110110 (tương đương 0x36 trong hệ thập lục phân) được lặp lại 64 lần để tạo thành một khối 512 bits.
- $\text{opad}$ : Lớp đệm ngoài (outer pad), bao gồm chuỗi nhị phân 01011100 (tương đương 0x5C trong hệ thập lục phân) được lặp lại 64 lần để tạo thành một khối 512 bits.

### 2.3 SSL/TLS Alert Protocol

Alert Protocol cung cấp khuôn thức chuẩn để hai bên TLS thông báo cho nhau các lỗi, tình huống bất thường hoặc sự kiện đặc biệt trong quá trình thiết lập và duy trì phiên. Đây là cơ chế báo động nội bộ của TLS, không dùng cho giao tiếp ứng dụng mà dùng để quản lý trạng thái và phản ứng khi có vấn đề.

Alert Protocol phân loại các thông báo thành hai cấp độ nghiêm trọng:

- warning: Cảnh báo không nghiêm trọng; phiên TLS thường vẫn có thể tiếp tục. Ví dụ điển hình là close notify thông báo đóng kết nối lịch sự. Bên nhận close notify hiểu rằng phía kia sẽ không gửi thêm dữ liệu, nhưng bên nhận vẫn có thể gửi nốt dữ liệu đang chờ trước khi đóng.
- fatal: Lỗi nghiêm trọng; kết nối phải đóng ngay. Sau khi nhận fatal alert, bên nhận phải hủy session identifier và không được tái sử dụng session đó. Ví dụ: bad record mac, handshake failure, decrypt error.

### 2.4 SSL/TLS Change Cipher Spec Protocol

Giao thức ChangeCipherSpec sở hữu cấu trúc đơn giản nhất, thường chỉ bao gồm một thông điệp có dung lượng đúng 1 byte. Chức năng duy nhất của nó là báo hiệu sự thay đổi về trạng thái hoạt động của các thuật toán mật mã. Khi một thực thể mạng phát đi thông điệp ChangeCipherSpec, nó đang chính thức thông báo cho thực thể đối tác rằng: bắt đầu từ thời điểm này trở đi, mọi gói tin được gửi từ phía nó sẽ hoàn toàn được mã hóa và bảo vệ bằng các thuật toán, cũng như các khóa phiên đã được thiết lập thành công trong quá trình Handshake trước đó.

## **2.5 SSL/TLS Handshake Protocol**

### ***2.5.1 Tổng quan về SSL/TLS Handshake Protocol***

Handshake Protocol là giao thức phức tạp nhất và tiêu tốn nhiều tài nguyên nhất trong bộ SSL/TLS, chịu trách nhiệm thiết lập các nền tảng mật mã học trước khi bắt kỳ một byte dữ liệu ứng dụng nào được phép truyền đi.

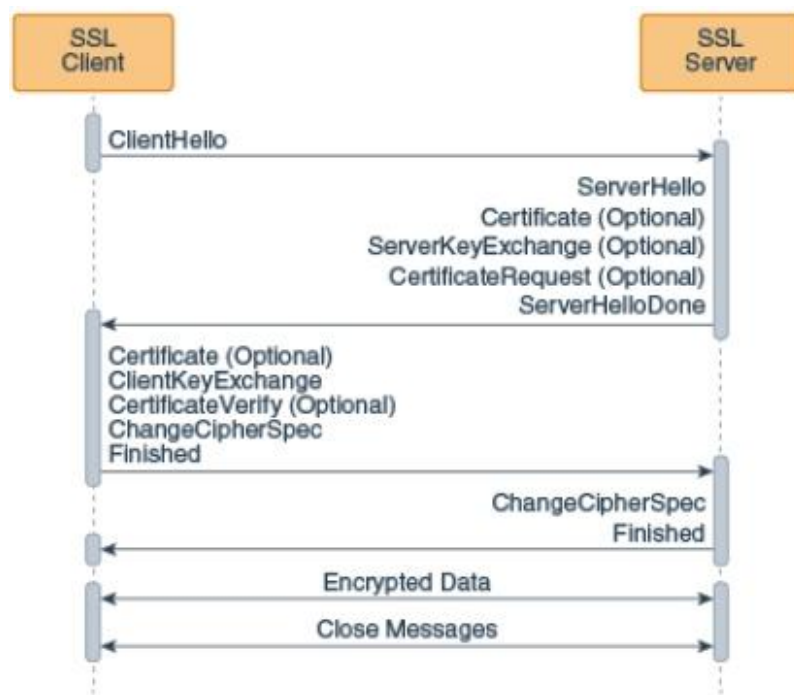
Handshake Protocol Quá trình bắt tay không diễn ra liên tục mà chỉ xuất hiện trong các thời điểm then chốt để thiết lập hoặc khôi phục sự tin tưởng:

- **Kết nối ban đầu (Initial Connection):** Khi người dùng truy cập một website HTTPS lần đầu tiên trong một phiên làm việc, trình duyệt phải thực hiện một quy trình bắt tay đầy đủ (Full Handshake).
- **Hết hạn phiên làm việc (Session Expiration):** Để đảm bảo an ninh, các khóa mật mã không tồn tại vĩnh viễn. Khi một phiên cũ hết hạn, client buộc phải thực hiện bắt tay mới gọi là Khôi phục phiên (Session Resumption)
- **Ngắt kết nối và Timeout:** Các sự cố mạng hoặc thời gian chờ thường xảy ra sau khoảng 60-120 giây không hoạt động. Khi kết nối bị đóng, yêu cầu tiếp theo sẽ kích hoạt một quy trình bắt tay mới để dựng lại đường ống bảo mật.
- **Cập nhật chứng chỉ:** Nếu máy chủ thay đổi hoặc gia hạn chứng chỉ SSL/TLS, client sẽ thực hiện bắt tay mới để xác minh tính hợp lệ của chứng chỉ mới này với các Cơ quan cấp chứng chỉ (CA), đảm bảo không kết nối với máy chủ mạo danh sử dụng chứng chỉ cũ đã bị thu hồi.

### ***2.5.2 Quá trình Bắt tay đầy đủ***

Trong TLS 1.2, quy trình bắt tay được thiết kế dựa trên mô hình trao đổi thông điệp tuần tự, yêu cầu 2 vòng khứ hồi (2-RTT) trước khi dữ liệu ứng dụng có thể được truyền tải. Điều này tạo ra độ trễ đáng kể, đặc biệt trên các mạng di động có độ trễ cao.





Hình 5 Quy trình Full Handshake

Quy trình bắt tay TLS 1.2 bao gồm các bước sau:

1. Trao đổi thông điệp chào hỏi (hello messages): Để thống nhất về các thuật toán, trao đổi các giá trị ngẫu nhiên và kiểm tra việc khôi phục phiên.
2. Xác thực Server và trao đổi khóa: Server chứng minh danh tính và cung cấp các tham số cần thiết để tạo khóa.
3. Xác thực Client và trao đổi khóa: Client chứng minh danh tính và thực hiện các bước để tạo premaster secret và master secret.
4. Hoàn tất và chuyển giao: Xác nhận mọi thứ đã sẵn sàng và chuyển sang chế độ mã hóa.

#### 2.5.2.1 ClientHello

Khi client kết nối lần đầu, bắt buộc phải gửi ClientHello như thông điệp đầu tiên. Cấu trúc của một ClientHello đầy đủ có các trường:

- client\_version: phiên bản TLS client muốn dùng; nên là phiên bản mới nhất client hỗ trợ.
- random: giá trị random do client tạo ra tổng cộng 32 bytes:
  - 4 bytes lưu trữ thời gian thực tại thời điểm gửi thông điệp, giúp ngăn chặn việc ử dụng lại các tham số ngẫu nhiên từ các phiên cũ.
  - 28 bytes được tạo ra từ bộ sinh số Cryptographically Strong Pseudo-Random Number Generator, đảm bảo mỗi lần bắt tay là duy nhất.
- session\_id: một chuỗi byte có độ dài biến thiên (từ 0 đến 32 bytes), cho phép tái sử dụng các tham số bảo mật đã thỏa thuận trước đó.



- `cipher_suites`: danh sách tùy chọn thuật toán bảo mật mà client hỗ trợ, gồm: trao đổi khóa (Key Exchange), mã hóa khối (Bulk Encryption), xác thực thông điệp (MAC), hàm giả ngẫu nhiên (PRF).
- `compression_methods`: danh sách thuật toán nén dữ liệu trước khi mã hóa để tiết kiệm băng thông.
- `extensions`: client có thể yêu cầu các trường mở rộng như Server Name Indication (SNI) để chỉ định tên miền, `signature_algorithms` để thông báo thuật toán chữ ký được hỗ trợ,...

#### 2.5.2.2 *ServerHello*

Server sẽ gửi *ServerHello* ngay sau khi nhận và xử lý xong *ClientHello* nếu tìm được tập thuật toán phù hợp, nếu không sẽ trả về thông báo lỗi và ngắt kết nối. Cấu trúc của một *ServerHello* gồm các trường:

- `server_version`: là phiên bản cao nhất mà máy chủ hỗ trợ, đồng thời phải thấp hơn hoặc bằng phiên bản mà máy khách đề xuất.
- `random`: tương tự như *ClientHello* nhưng do server tạo và phải độc lập với *ClientHello.random*.
- `session_id`: một chuỗi byte có độ dài biến thiên (từ 0 đến 32 bytes).
- `cipher_suite`: cipher suite server chọn từ danh sách client
- `compression_method`: phương pháp nén server chọn (với resume thì là của session cũ).
- `extensions`: chỉ các extension mà client đã đề xuất mới được xuất hiện trong *ServerHello*.

#### 2.5.2.3 *ServerCertificate*

Thông điệp *Server Certificate* là bước để máy chủ chứng minh danh tính của máy chủ với máy khách. Thông điệp được gửi ngay sau *ServerHello*, bắt buộc với hầu hết các bộ Cipher Suite hiện đại (như RSA, ECDHE\_RSA,...). Máy chủ sẽ gửi một chuỗi chứng chỉ gồm:

- Chứng chỉ của chính máy chủ (End-entity certificate)
- Các chứng chỉ trung gian (Intermediate CAs); mỗi chứng chỉ sau sẽ ký xác nhận cho chứng chỉ ngay trước nó
- Chứng chỉ gốc (Root CA): có thể được lược bỏ để tiết kiệm băng thông vì Client thường đã lưu sẵn trong Trust Store của hệ điều hành hoặc trình duyệt.

Chứng chỉ được truyền đi dưới dạng dữ liệu thô ASN.1 DER encoding.

#### 2.5.2.4 *ServerKeyExchange*

Trong các phương thức trao đổi khóa hiện đại như DHE (Diffie-Hellman Ephemeral) hoặc ECDHE, khóa công khai nằm trong chứng chỉ của Server chỉ có tác dụng xác thực danh tính, chứ không dùng để mã hóa khóa. Vì vậy, server cần gửi thêm thông điệp này để chuyển cho client các tham số tạm thời để client có đủ dữ liệu để tính toán ra premaster secret. Premaster secret là giá trị bí mật ban đầu dùng để sinh các khóa mã hóa cho phiên kết nối.

#### 2.5.2.5 Certificate Request

Thông điệp Certificate Request là bước mà máy chủ thực hiện khi muốn xác thực danh tính của người dùng (Client Authentication), được gửi theo sau ServerKeyExchange (nếu có) hoặc sau Certificate của Server. Cấu trúc của một Certificate Request gồm các trường:

- `certificate_types`: Danh sách các loại chứng chỉ mà Server có thể xử lý (ví dụ: `rsa_sign`, `ecdsa_sign`).
- `supported_signature_algorithms`: (Chỉ có từ TLS 1.2) Danh sách các cặp Hash/Signature mà Server có khả năng xác minh. Client phải chọn một thuật toán trong danh sách này để ký thông điệp CertificateVerify sau đó.
- `certificate_authorities`: Danh sách tên các Nhà cung cấp chứng chỉ (CA) mà Server tin tưởng. Client sẽ dựa vào đây để chọn ra chứng chỉ phù hợp trong kho lưu trữ của mình

#### 2.5.2.6 ServerHelloDone

ServerHelloDone là thông điệp báo hiệu sự kết thúc của pha phản hồi từ phía Server. Thông điệp có cấu trúc rỗng, dù không chứa dữ liệu nhưng bản thân việc xuất hiện của loại thông điệp này là tín hiệu điều khiển bắt buộc. Sau khi gửi thông điệp này, Server sẽ dừng và đợi phản hồi từ Client.

#### 2.5.2.7 ClientCertificate

ClientCertificate là thông điệp đầu tiên mà Client có thể gửi sau khi nhận được thông điệp ServerHelloDone. Thông điệp này chỉ được gửi nếu Server yêu cầu một chứng chỉ thông qua CertificateRequest. Server sẽ sử dụng nó để:

- Lấy khóa công khai từ chứng chỉ vừa gửi để giải mã/xác minh chữ ký trong CertificateVerify
- Kết hợp với khóa bí mật của Server để tính toán ra premaster secret.

Chứng chỉ phải phù hợp với thuật toán trao đổi khóa của bộ Cipher Suite đã thỏa thuận và bất kỳ phần extensions nào đã thương lượng.

#### 2.5.2.8 ClientKeyExchange

ClientKeyExchange là thông điệp chứa dữ liệu cần thiết để cả Client và Server thiết lập premaster secret. Nội dung của thông điệp thay đổi tùy theo phương thức trao đổi khóa đã chọn ở ServerHello:

- Với RSA: Client tạo một chuỗi 48 byte ngẫu nhiên, mã hóa nó bằng khóa công khai của Server và gửi đi.
- Với DH: Client gửi khóa công khai tạm thời của mình để Server có thể tính toán ra bí mật chung dựa trên các tham số đã trao đổi trước đó.

Sau thông điệp này, cả hai bên có đủ nguyên liệu để sinh ra master secret. Đồng thời, đảm bảo rằng dù kẻ tấn công có bắt được toàn bộ gói tin bắt tay, chúng cũng không thể biết được bí mật bên trong nếu không có khóa bí mật của Server (RSA) hoặc không giải được bài toán logarit rời rạc (trong DH).

### 2.5.2.9 CertificateVerify

Thông điệp Certificate Verify chỉ xuất hiện khi máy chủ yêu cầu xác thực danh tính máy khách thông qua chứng chỉ số (Certificate Request).

Mục đích của Certificate Verify là cung cấp bằng chứng xác thực rằng máy khách thực sự sở hữu khóa bí mật (Private Key) tương ứng với chứng chỉ công khai đã gửi. Vai trò của nó bao gồm:

- Xác thực thực thể (Authentication): Ngăn chặn các cuộc tấn công mạo danh bằng cách yêu cầu máy khách ký lên một bản tóm tắt (hash) chứa toàn bộ lịch sử các thông điệp bắt tay trước đó (từ ClientHello đến ngay trước thông điệp này).
- Đảm bảo tính toàn vẹn: Do chữ ký bao phủ toàn bộ các thông điệp đã trao đổi, bất kỳ sự can thiệp hay sửa đổi nào từ bên thứ ba (MITM) vào các gói tin trước đó đều sẽ khiến chữ ký bị sai lệch và quá trình bắt tay sẽ bị hủy bỏ ngay lập tức.
- Ràng buộc phiên: Chữ ký này là duy nhất cho mỗi phiên kết nối vì nó bao gồm các giá trị ngẫu nhiên (ClientRandom và ServerRandom) của phiên đó, giúp chống lại các cuộc tấn công phát lại (Replay Attacks).

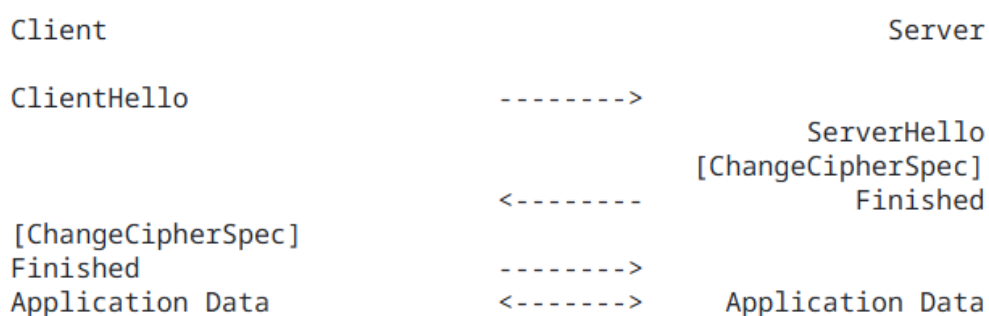
### 2.5.2.10 Finished

Thông điệp Finished luôn được gửi ngay sau thông điệp ChangeCipherSpec để xác nhận rằng quá trình trao đổi khóa và xác thực đã thành công. Thông điệp Finished là thông điệp đầu tiên được bảo vệ (mã hóa) bằng các thuật toán, khóa và bí mật vừa mới thỏa thuận xong. Ý nghĩa của thông điệp:

- Xác nhận tính toàn vẹn: Kiểm tra xem toàn bộ quá trình bắt tay (từ ClientHello) có bị bên thứ ba can thiệp hay không thông qua việc đối chiếu mã băm `verify_data`.
- Kích hoạt truyền dữ liệu: Việc trao đổi thành công thông điệp Finished từ cả hai phía đánh dấu thời điểm kênh truyền bảo mật chính thức được thiết lập, cho phép dữ liệu ứng dụng bắt đầu được truyền đi.

## 2.5.3 Cơ chế khôi phục phiên

Cơ chế khôi phục phiên (Session Resumption) trong giao thức TLS 1.2 được thiết kế để tối ưu hóa hiệu suất bằng cách bỏ qua các bước đàm phán và trao đổi khóa bất đối xứng tiêu tốn nhiều tài nguyên của quá trình bắt tay toàn phần (full handshake). Bằng cách này, máy khách và máy chủ có thể tái sử dụng các thông số bảo mật và khóa phiên đã được thiết lập từ kết nối trước đó, giúp giảm đáng kể độ trễ.



### *Hình 6 Cơ chế khôi phục phiên*

Trong TLS 1.2, có hai phương thức tiêu chuẩn để khôi phục phiên: Session ID (máy chủ lưu trữ) và Session Ticket (máy khách lưu trữ).

#### *2.5.3.1 Khôi phục phiên sử dụng Session ID*

Với cơ chế này, máy chủ chịu trách nhiệm lưu trữ các thông số của phiên làm việc trong bộ nhớ cache cục bộ và gán cho nó một mã định danh duy nhất (Session ID), sau đó gửi cho máy khách.

- Khi muốn tái kết nối, máy khách sẽ gửi lại Session ID này ngay trong thông điệp Client Hello.
- Nếu máy chủ tìm thấy Session ID hợp lệ trong bộ nhớ cache của mình, nó sẽ trả lời bằng chính ID đó và ngay lập tức chuyển sang quá trình bắt tay rút gọn (abbreviated handshake). Nếu không tìm thấy, hai bên buộc phải quay lại quy trình đàm phán khóa từ đầu.

Nhược điểm của cơ chế này là đòi hỏi máy chủ phải tiêu tốn lượng lớn bộ nhớ (RAM) để duy trì cache và gây ra nhiều thách thức về việc đồng bộ hóa dữ liệu khi hệ thống web sử dụng kiến trúc cân bằng tải (load balancer) với nhiều máy chủ vật lý khác nhau.

#### *2.5.3.2 Khôi phục phiên sử dụng Session Ticket*

Session Ticket là một phần mở rộng ra đời nhằm khắc phục vấn đề quá tải bộ nhớ của Session ID bằng cách chuyển toàn bộ gánh nặng lưu trữ sang phía máy khách.

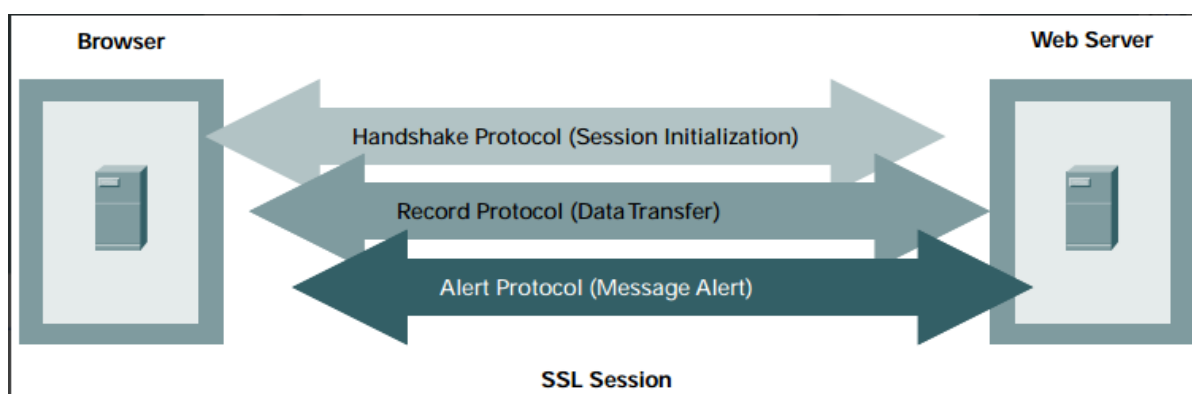
- Ở vòng kết nối đầu tiên, sau khi đàm phán xong, máy chủ sẽ đóng gói toàn bộ trạng thái phiên làm việc (bao gồm cả khóa master bí mật) và mã hóa khối dữ liệu này bằng một khóa bảo mật riêng mà chỉ máy chủ sở hữu. Khối dữ liệu mã hóa này được gọi là "Ticket" và được gửi cho máy khách để lưu trữ.
- Ở lần kết nối sau, máy khách gửi kèm Ticket này. Máy chủ chỉ việc dùng khóa riêng của mình để giải mã Ticket, lấy ra các thông số kết nối và khôi phục ngay phiên giao dịch mà không cần phải tra cứu bộ nhớ nội bộ.

## CHƯƠNG 3. CƠ CHẾ HOẠT ĐỘNG CỦA GIAO THỨC SSL/TLS

### 3.1 Tổng quan

Cơ chế hoạt động của TLS 1.2 được thiết kế dựa trên một bộ giao thức phân lớp phức tạp, đảm nhận các nhiệm vụ từ đàm phán thông số, xác thực danh tính cho đến mã hóa dữ liệu. Quá trình vận hành này được chia thành các giai đoạn sau:

1. Giai đoạn Đàm phán sơ bộ và xác thực
2. Giai đoạn Dẫn xuất Khóa
3. Giai đoạn Kích hoạt Bảo mật
4. Giai đoạn Truyền dữ liệu



Hình 7 Cơ chế hoạt động của SSL/TLS

### 3.2 Giai đoạn Đàm phán sơ bộ và xác thực

Khi kết nối mạng TCP vừa hình thành, các thuật toán mã hóa, hàm băm bảo vệ toàn vẹn và cơ chế nén của tầng Record bắt buộc được khởi tạo ở trạng thái null. Mọi thông điệp đàm phán xuất phát từ Giao thức Handshake (như Client Hello, Server Hello) đều được Giao thức Record tiếp nhận, phân mảnh và bọc thêm phần Tiêu đề (Header).

Vì băng chuyền đang chạy ở trạng thái không mã hóa, không xác thực, nên các thông số như: danh sách bộ mã hóa của trình duyệt, chứng chỉ số của máy chủ hay các chuỗi số ngẫu nhiên đều ở dạng bản rõ. Tuy nhiên, pre-master secret sẽ được bảo vệ bằng cách mã hóa bằng khóa công khai (RSA) của máy chủ hoặc được cả hai bên tự tính toán ngầm thông qua nền tảng toán học DH. Vì vậy, kẻ tấn công nghe lén không thể tính toán ra khóa giải mã cuối cùng.

### 3.3 Giai đoạn Dẫn xuất Khóa

Ngay khi bước trao đổi thông điệp ClientKeyExchange kết thúc, quá trình Dẫn xuất khóa (Key Derivation) sẽ lập tức được kích hoạt.

- Tính toán khóa chính: Do độ dài của Premaster Secret thay đổi tùy thuộc vào thuật toán trao đổi khóa (ví dụ: RSA cho ra độ dài khác với Diffie-Hellman), hệ thống cần chuẩn hóa nó thành một chuỗi có độ dài cố định. Cả hai bên sẽ sử dụng một Hàm giả ngẫu nhiên (Pseudorandom Function - PRF), thường dựa trên thuật toán HMAC-SHA256, để trộn các nguyên liệu lại theo công thức:  $\text{Master Secret} = \text{PRF}(\text{Premaster Secret}, \text{"master"})$

secret", Client Random + Server Random) . Kết quả của hàm này luôn cho ra một master secret có độ dài chuẩn xác là 48 bytes. Việc đưa cả Client Random và Server Random vào công thức giúp ràng buộc chặt chẽ Khóa chính này vào đúng phiên giao dịch hiện tại.

- Mở rộng khóa: Master secret được sử dụng để mở rộng thành một khối dữ liệu khóa không lờ (key block) thông qua hàm PRF: Key Block = PRF(Master Secret, "key expansion", Server Random + Client Random) . Hàm PRF sẽ liên tục chạy cho đến khi sinh đủ số lượng byte cần thiết cho các thuật toán mã hóa.
- Phân rã Khối khóa: Để đảm bảo an toàn tuyệt đối và ngăn chặn Tấn công phản xạ, TLS không dùng một khóa cho cả hai chiều tải lên và tải xuống. Khối Key Block ở Giai đoạn 2 sẽ được cắt ra thành 6 phần (khóa) chuyên biệt theo đúng thứ tự sau:
  1. Client Write MAC Key: Khóa dùng để Client tạo mã xác thực toàn vẹn (MAC) cho gói tin gửi đi.
  2. Server Write MAC Key: Khóa dùng để Server tạo mã xác thực toàn vẹn (MAC) cho gói tin gửi đi.
  3. Client Write Key: Khóa đối xứng (ví dụ AES) dùng để Client mã hóa dữ liệu.
  4. Server Write Key: Khóa đối xứng dùng để Server mã hóa dữ liệu.
  5. Client Write IV: Vector khởi tạo (Initialization Vector) dùng cho các thuật toán mã hóa khối của Client (nếu cần).
  6. Server Write IV: Vector khởi tạo của Server (nếu cần).

### 3.4 Giai đoạn Kích hoạt Bảo mật

Khi đã có khóa, hai bên gửi thông điệp ChangeCipherSpec để ra lệnh cho Giao thức Record cập nhật trạng thái hoạt động. Kể từ khoảnh khắc này, băng chuyền Record chính thức được kích hoạt chế độ bảo vệ. Giao thức Handshake gửi thông điệp Finished (gói tin đầu tiên được mã hóa) để kiểm tra chéo xem quá trình đàm phán trước đó có bị hacker can thiệp hay không.

### 3.5 Giai đoạn Truyền dữ liệu

Lúc này, khi ứng dụng cấp cao (ví dụ: trình duyệt web gửi yêu cầu HTTP hoặc người dùng nhập mật khẩu) muốn gửi dữ liệu, luồng thông tin này sẽ được đẩy xuống Giao thức Record (Record Protocol). Tại đây, "băng chuyền" Record đã được kích hoạt chế độ bảo mật sẽ thực thi một chuỗi quy trình khép kín gồm các bước đã được nêu tại mục 3.2.2. Trong suốt vòng đời của 4 bước trên, Giao thức Alert vẫn luôn chạy ngầm để giám sát. Nếu quá trình giải mã thất bại hoặc mã MAC không khớp do gói tin bị can thiệp, nó sẽ lập tức phát cảnh báo lỗi và đánh sập đường truyền.

## CHƯƠNG 4. CÁC LỖ HỔNG VÀ CÁC DẠNG TẤN CÔNG VÀO SSL/TLS

### 4.1 Mở đầu

Dựa trên cơ chế hoạt động và bản chất của điểm yếu, các mối đe dọa nhắm vào hệ thống TLS có thể được phân loại một cách hệ thống thành hai nhóm: các cuộc tấn công hạ cấp giao thức, tấn công vào điểm yếu toán học. Việc phân tích và nhận diện rõ từng nhóm đe dọa không chỉ giúp hiểu sâu bản chất rủi ro mà còn là cơ sở quan trọng để đề xuất các biện pháp phòng vệ hiệu quả và bền vững.

### 4.2 Tấn công hạ cấp (Downgrade Attacks)

#### 4.2.1 Mô tả

Tấn công hạ cấp là hình thức tấn công chủ động trong đó kẻ tấn công đặt mình ở vị trí trung gian giữa máy khách và máy chủ, can thiệp vào quá trình thương lượng giao thức (protocol negotiation) nhằm ép cả hai bên chấp nhận sử dụng một phiên bản giao thức hoặc bộ mã hóa lỗi thời, kém an toàn hơn.

Nguyên lý cốt lõi của nhóm tấn công này xoay quanh cơ chế tương thích ngược. Trong quá trình thiết lập kết nối bảo mật, máy khách và máy chủ thực hiện một bước thương lượng để thống nhất về phiên bản giao thức sẽ sử dụng. Thông thường, phiên bản cao nhất mà cả hai bên cùng hỗ trợ sẽ được chọn. Tuy nhiên, khi kết nối thất bại vì bất kỳ lý do nào, nhiều triển khai được thiết kế để tự động thử lại với phiên bản thấp hơn nhằm đảm bảo kết nối có thể được thiết lập. Kẻ tấn công lợi dụng hành vi dự phòng này bằng cách cố ý gây ra lỗi kết nối, từ đó kéo phiên bản giao thức đang được sử dụng xuống mức mà kẻ tấn công mong muốn.

Để một cuộc tấn công hạ cấp có thể xảy ra, hai điều kiện cơ bản phải được thỏa mãn đồng thời:

- Điều kiện thứ nhất là hệ thống mục tiêu phải vẫn còn hỗ trợ ít nhất một phiên bản giao thức cũ chứa lỗ hổng đã biết. Nếu máy chủ và máy khách đã hoàn toàn loại bỏ các giao thức lỗi thời, kẻ tấn công dù có thể ép hạ cấp cũng không còn đích để khai thác.
- Điều kiện thứ hai là cơ chế thương lượng giao thức không có biện pháp bảo vệ chống lại việc hạ cấp bị áp đặt từ bên ngoài, tức là không có cách nào để hai bên xác minh rằng quá trình thương lượng diễn ra trung thực mà không bị can thiệp.

#### 4.2.2 Hậu quả

Các cuộc tấn công hạ cấp nhắm vào cơ chế đàm phán của giao thức TLS mang lại những hệ lụy đặc biệt nghiêm trọng sau:

- **Truy cập dữ liệu trái phép:** Kẻ tấn công có thể ép hệ thống sử dụng các chế độ kém an toàn hơn, từ đó dễ dàng giành quyền truy cập trái phép vào các dữ liệu nhạy cảm.
- **Chiếm đoạt phiên (Session Hijacking):** Các cuộc tấn công hạ cấp có thể dẫn đến việc chiếm đoạt phiên, cho phép kẻ tấn công giành quyền kiểm soát phiên làm việc của người dùng và có khả năng đánh cắp được các token xác thực.

- **Tăng nguy cơ bị nghe lén:** Các giao thức mã hóa yếu giúp kẻ tấn công dễ dàng chặn bắt và giải mã thông tin liên lạc hơn, dẫn đến các vụ rò rỉ dữ liệu.
- **Đe dọa đến truyền thông an toàn:** Việc ép buộc sử dụng các giao thức cũ, chứa nhiều lỗ hổng sẽ làm tổn hại đến tính toàn vẹn và tính bảo mật của các luồng giao tiếp an toàn.
- **Lộ thông tin nhạy cảm:** Các cuộc tấn công hạ cấp có thể dẫn đến việc truyền tải thông tin nhạy cảm ở các chế độ không được mã hóa, khiến chúng dễ dàng lọt vào tay kẻ tấn công.

#### 4.2.3 Biện pháp phòng chống

Một số biện pháp để chống lại cuộc tấn công hạ cấp TLS:

- **Vô hiệu hóa các giao thức lỗi thời:** Cách triệt để nhất là cấu hình hệ thống từ chối hoàn toàn các phiên bản đã lỗi thời và nhiều lỗ hổng như SSL 3.0, TLS 1.0 hay TLS 1.1.
- **Sử dụng cơ chế TLS\_FALLBACK\_SCSV (trong các trường hợp không thể vô hiệu hóa hoàn toàn các giao thức cũ):** Khi máy khách chủ động thử lại kết nối với phiên bản thấp hơn sau một lần thất bại, nó sẽ chèn giá trị TLS\_FALLBACK\_SCSV vào thông điệp ClientHello để báo cho máy chủ biết đây là một lần fallback. Nếu máy chủ thực tế hỗ trợ phiên bản cao hơn, nó sẽ từ chối kết nối và trả về lỗi `inappropriate_fallback`. Cơ chế này giúp ngăn chặn kẻ tấn công giả mạo lỗi để ép hạ cấp, đảm bảo quá trình bắt tay chỉ được thiết lập khi phù hợp và an toàn.
- **Cập nhật phần mềm thường xuyên:** Giữ cho toàn bộ phần mềm, hệ điều hành và thư viện mật mã luôn được cập nhật các bản vá bảo mật mới nhất nhằm giảm thiểu rủi ro từ các lỗ hổng đã biết.
- **Thực thi các chính sách bảo mật nghiêm ngặt:** Sử dụng chính sách bảo mật như HSTS để ra lệnh cho trình duyệt của người dùng luôn phải dùng đường truyền bảo mật HTTPS mỗi khi truy cập vào hệ thống.
- **Sử dụng các giao thức hiện đại:** Sử dụng phiên bản TLS mới nhất, như TLS 1.3, vốn đã được thiết kế tích hợp sẵn các cơ chế bảo vệ (như giá trị cảnh báo downgrade trong luồng `ServerHello.Random`) để chống lại các cuộc tấn công hạ cấp.

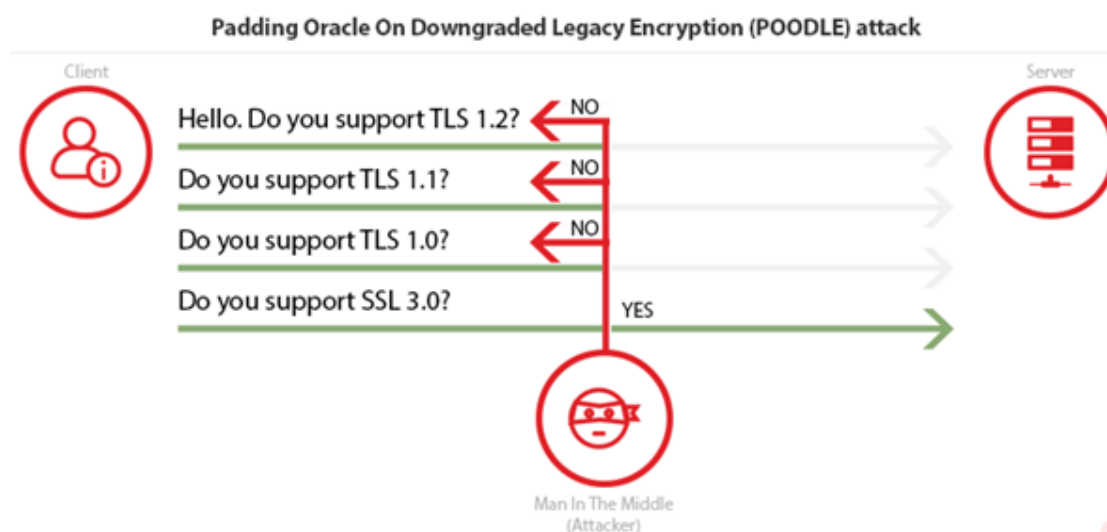
#### 4.2.4 Cuộc tấn công POODLE

POODLE khai thác việc hệ thống vẫn còn duy trì hỗ trợ SSL 3.0 như một phương án dự phòng. Mặc dù TLS đã ra đời và được triển khai rộng rãi, hầu hết các máy chủ và trình duyệt vào thời điểm lỗ hổng được công bố vẫn giữ SSL 3.0 trong danh sách giao thức được hỗ trợ để đảm bảo tương thích với các hệ thống cũ.

Cuộc tấn công bắt đầu từ việc kẻ tấn công chặn và can thiệp vào quá trình bắt tay (handshake) SSL/TLS giữa máy khách và máy chủ. Trong giai đoạn này, máy khách gửi đề xuất sử dụng phiên bản TLS mới nhất mà nó hỗ trợ. Kẻ tấn công lập tức gây ra lỗi kết nối, khiến quá trình bắt tay thất bại. Khi đó, máy khách — theo cơ chế fallback được thiết kế sẵn — sẽ tự động thử lại kết nối với phiên bản giao thức thấp hơn. Kẻ tấn công tiếp tục lặp lại việc phá vỡ kết nối sau mỗi lần thử, cho đến khi máy khách hạ xuống đúng SSL 3.0. Tại



thời điểm đó, kẻ tấn công để kết nối được thiết lập thành công, và từ đây toàn bộ giao tiếp diễn ra trên nền SSL 3.0 với tất cả các điểm yếu vốn có của nó.



Hình 8 Kẻ tấn công liên tục phá vỡ kết nối đến khi máy khách hạ xuống SSL 3.0

Biện pháp phòng chống căn bản và triệt để nhất là vô hiệu hóa hoàn toàn hỗ trợ SSL 3.0 trên cả phía máy chủ lẫn phía máy khách. Khi SSL 3.0 không còn được hỗ trợ, kẻ tấn công dù có thể gây ra lỗi kết nối liên tục cũng không thể ép hệ thống hạ cấp xuống giao thức này, vì nó đơn giản là không tồn tại trong danh sách các tùy chọn khả dụng.

Trong các trường hợp không thể vô hiệu hóa hoàn toàn SSL 3.0 ngay lập tức, cơ chế TLS\_FALLBACK\_SCSV (Signaling Cipher Suite Value) cung cấp một lớp bảo vệ bổ sung nhằm ngăn chặn việc hạ cấp giao thức không cần thiết.

Nguyên lý hoạt động của cơ chế này như sau: khi máy khách thực hiện fallback — tức là chủ động thử lại kết nối với phiên bản giao thức thấp hơn sau một lần thất bại — nó đưa thêm một giá trị đặc biệt có tên TLS\_FALLBACK\_SCSV vào danh sách cipher suite trong thông điệp ClientHello. Giá trị này hoạt động như một tín hiệu thông báo cho máy chủ biết rằng đây là một lần thử lại sau khi thất bại, chứ không phải là kết nối bình thường từ một máy khách chỉ hỗ trợ phiên bản giao thức thấp.

Khi nhận được tín hiệu này, nếu máy chủ biết rằng nó hỗ trợ phiên bản giao thức cao hơn phiên bản mà máy khách đang yêu cầu trong lần thử lại đó, máy chủ sẽ từ chối kết nối và trả về cảnh báo lỗi `inappropriate_fallback`. Điều này ngăn chặn kịch bản kẻ tấn công giả mạo lỗi kết nối để ép hạ cấp, vì ngay cả khi máy khách đã bị lừa để yêu cầu phiên bản thấp hơn, máy chủ vẫn từ chối chấp nhận kết nối đó khi nhận thấy sự bất hợp lý. Cả hai bên đều có khả năng sử dụng phiên bản cao hơn, do đó không có lý do chính đáng nào cho việc hạ cấp, và kết nối sẽ không được thiết lập cho đến khi quá trình bắt tay diễn ra trung thực.

## 4.3 Tấn công vào điểm yếu toán học

### 4.3.1 Mô tả

Tấn công vào điểm yếu toán học và thiết kế giao thức là lỗ hổng khai thác trực tiếp các điểm yếu toán học vốn có trong thiết kế của các bộ mã hóa (cipher suites) lỗi thời của giao

thức TLS. Khi sự phát triển vượt bậc của năng lực tính toán hoặc các kỹ thuật phân tích mật mã tiên tiến, bản thân thuật toán sẽ không còn đảm bảo tính bảo mật như thiết kế ban đầu, dữ liệu được bảo vệ bởi các thuật toán này trở nên có thể giải mã được dù kết nối vẫn đang hoạt động bình thường ở tầng giao thức.

### 4.3.2 Hậu quả

Hậu quả trực tiếp là kẻ tấn công có thể giải mã các luồng dữ liệu một cách âm thầm để đánh cắp các thông tin cực kỳ nhạy cảm như cookie phiên, mật khẩu, token xác thực,... Mức độ nghiêm trọng hơn, kẻ gian có thể thao túng chữ ký số và xác thực, cho phép chúng mạo danh máy chủ hợp lệ và kiểm soát hoàn toàn thông tin giao tiếp (Man-in-the-Middle),...

### 4.3.3 Biện pháp phòng chống

- **Sử dụng giao thức hiện đại:** Việc chuyển đổi sang phiên bản TLS 1.3 là giải pháp phòng vệ triệt để nhất. TLS 1.3 được thiết kế lại để loại bỏ hoàn toàn các chế độ mã hóa cũ dễ bị tấn công (như chế độ CBC), bắt buộc sử dụng các thuật toán mã hóa xác thực mạnh (AEAD) và loại bỏ các nền tảng trao đổi khóa dễ bị xâm phạm.
- **Vô hiệu hóa các thuật toán mã hóa lỗi thời:** nếu bắt buộc phải duy trì TLS 1.2, cần cấu hình lại cipher suites để loại bỏ các thuật toán yếu (RC4, SHA-1, blowfish, 3DES,...), ưu tiên sử dụng các bộ mã mạnh như ECDHE + AES-GCM (hoặc ChaCha20-Poly1305).

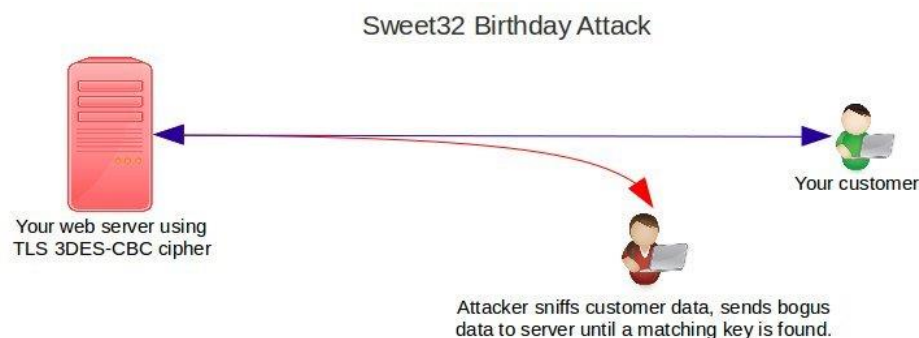
### 4.3.4 Một số cuộc tấn công

| Cuộc tấn công                                           | Mô tả                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BEAST (Browser Exploit Against SSL/TLS)                 | Kẻ tấn công nhắm vào các bộ mã hơas khối trong TLS 1.0 sử dụng chế độ CBC (Cipher Block Chaining). Sau đó sử dụng kỹ thuật "Chosen Boundary Attack" để dự đoán Vector khởi tạo (IV) của khối dữ liệu tiếp theo. Qua đó, có thể giải mã từng byte của các cookie HTTP nhạy cảm, cho phép chiếm đoạt phiên làm việc của người dùng. |
| POODLE (Padding Oracle On Downgraded Legacy Encryption) | Lợi dụng cách thức xử lý dữ liệu đệm (padding) không nghiêm ngặt của SSL 3.0. Kẻ tấn công thay đổi các byte đệm và quan sát phản ứng của máy chủ. Nếu máy chủ chấp nhận, kẻ tấn công có thể giải mã nội dung bên trong (thường là cookie) sau khoảng 256 lần thử cho mỗi byte.                                                    |
| FREAK (Factoring RSA Export Keys)                       | Kẻ tấn công đứng giữa (MITM) đánh lừa máy chủ sử dụng khóa RSA 512-bit yếu thay vì khóa mạnh. Với sức mạnh tính toán hiện nay, kẻ tấn công có thể phân tích nhân tử (factoring) khóa này nhanh chóng để tìm ra khóa bí mật và giải mã toàn bộ dữ liệu.                                                                            |
| Logjam                                                  | Tương tự như FREAK nhưng nhắm vào thuật toán DH. Kẻ tấn công ép kết nối hạ cấp xuống mức "Export Diffie-Hellman" (512-bit). Do nhiều máy chủ sử dụng chung các số nguyên tố mặc định, kẻ tấn công có thể tính toán trước                                                                                                          |

|         |                                                                                                                                                                                                                                                                                                                                                                 |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         | các bảng dữ liệu để bẻ khóa hàng loạt kết nối trong thời gian thực.                                                                                                                                                                                                                                                                                             |
| SWEET32 | Kẻ tấn công nhằm vào các bộ mã hóa sử dụng kích thước khối 64 bits (3DES hoặc Blowfish). Khi một kết nối kéo dài và truyền đi một lượng lớn dữ liệu (khoảng 785 GB), xác suất xảy ra xung đột khối (collision) tăng cao do kích thước khối nhỏ. Kẻ tấn công có thể lợi dụng sự xung đột này để khôi phục các đoạn văn bản gốc (plaintext) từ dữ liệu đã mã hóa. |

#### 4.3.5 Cuộc tấn công SWEET32

SWEET32 là tên gọi của một cuộc tấn công nhằm vào các thuật toán mã hóa khối có độ dài khối ngắn 64-bit, điển hình là 3DES (Triple Data Encryption Standard) và Blowfish. Được công bố năm 2016, SWEET32 không khai thác lỗi trong hiện thực hóa hay điểm yếu của khóa, mà khai thác một giới hạn toán học cơ bản: khi kích thước không gian khối (block space) đủ nhỏ, xác suất xuất hiện hai khối bản mã giống nhau (collision) trong một phiên mã hóa dài sẽ tăng lên đến mức có thể khai thác được. Đây là biểu hiện trực tiếp của Birthday Paradox áp dụng vào lĩnh vực mật mã học.



Hình 9 Tấn công SWEET32

Với thuật toán mã hóa khối có kích thước khối  $n$  bit, không gian các giá trị khối bản mã có thể có là  $2^n$ . Theo nguyên lý Birthday Paradox, xác suất xuất hiện ít nhất một cặp va chạm (collision) — tức là hai khối bản rõ khác nhau tạo ra cùng một khối bản mã — đạt xấp xỉ 50% khi số lượng khối được mã hóa đạt đến ngưỡng  $2^{(n/2)}$ . Với thuật toán có kích thước khối 64-bit như 3DES, ngưỡng này là  $2^{32}$ , tương đương với khoảng 32 GB dữ liệu được mã hóa bằng cùng một khóa trong cùng một phiên.

Trong chế độ CBC, khi hai khối bản rõ  $P_1$  và  $P_2$  tạo ra cùng một khối bản mã  $C$  (tức là xảy ra va chạm), kẻ tấn công quan sát được điều này qua bản mã. Từ cấu trúc của CBC, có thể suy ra rằng giá trị XOR của hai khối bản rõ tương ứng bằng XOR của hai khối bản mã liền trước chúng — tất cả đều là các giá trị mà kẻ tấn công đã biết. Nếu kẻ tấn công biết nội dung của một trong hai khối bản rõ (ví dụ thông qua kiến thức về cấu trúc của yêu cầu HTTP), chúng có thể tính ngược ra nội dung của khối bản rõ còn lại. Quá trình này được

thực hiện theo từng cặp và chạm được phát hiện, dần dần khôi phục các phần của dữ liệu bí mật — thường là cookie phiên hoặc các header HTTP chứa thông tin xác thực.

Để thu thập đủ 32 GB dữ liệu được mã hóa trong một phiên TLS duy nhất — điều kiện cần để đạt ngưỡng va chạm — kẻ tấn công cần duy trì một kết nối dài và liên tục. Trong thực tế, điều này có thể đạt được bằng cách sử dụng mã JavaScript trong trình duyệt của nạn nhân để liên tục gửi yêu cầu HTTP đến máy chủ mục tiêu qua một kết nối HTTPS duy trì lâu dài, không bị ngắt. Mặc dù 32 GB là một lượng dữ liệu không nhỏ, thời gian cần thiết để truyền lượng dữ liệu này qua một kết nối tốc độ cao trong điều kiện thực tế — được ước tính khoảng vài chục giờ với kết nối trung bình — vẫn nằm trong ngưỡng thực tế của một cuộc tấn công có tính kiên nhẫn.

Điểm quan trọng cần nhấn mạnh là SWEET32 không phải là lỗ hổng phát sinh do sai sót triển khai hay cấu hình, mà là hệ quả tất yếu của việc tiếp tục sử dụng một thuật toán mà giới hạn toán học của nó đã không còn đủ an toàn trong điều kiện tốc độ truyền thông tin hiện đại. Khi 3DES được thiết kế, lượng dữ liệu 32 GB trong một phiên duy nhất là điều không tưởng; trong bối cảnh mạng tốc độ cao ngày nay, giới hạn đó không còn là rào cản thực sự.

Biện pháp phòng chống triệt để và được khuyến nghị là loại bỏ hoàn toàn 3DES khỏi danh sách các bộ mã hóa được hỗ trợ và thay thế bằng AES với độ dài khóa 128 hoặc 256-bit. AES sử dụng kích thước khối 128-bit, nâng ngưỡng va chạm Birthday lên  $2^{64}$  — tức là khoảng 17 tỷ tỷ GB dữ liệu — một mức hoàn toàn nằm ngoài tầm với của bất kỳ cuộc tấn công thực tế nào trong tương lai có thể dự kiến được.

Sự lựa chọn AES không chỉ giải quyết vấn đề kích thước khối mà còn mang lại nhiều lợi ích bảo mật bổ sung: AES được thiết kế với tính bảo mật chống lại nhiều lớp tấn công mật mã học hiện đại, được tối ưu hóa phần cứng trên hầu hết các nền tảng xử lý hiện tại thông qua tập lệnh AES-NI, và khi kết hợp với chế độ GCM tạo thành AES-GCM — một thuật toán AEAD — còn cung cấp đồng thời cả tính bảo mật lẫn tính xác thực dữ liệu trong một cơ chế duy nhất, loại bỏ toàn bộ họ tấn công liên quan đến thao túng bản mã mà không bị phát hiện.

## CHƯƠNG 5. ỨNG DỤNG CỦA GIAO THỨC SSL/TLS

### 5.1 Mở đầu

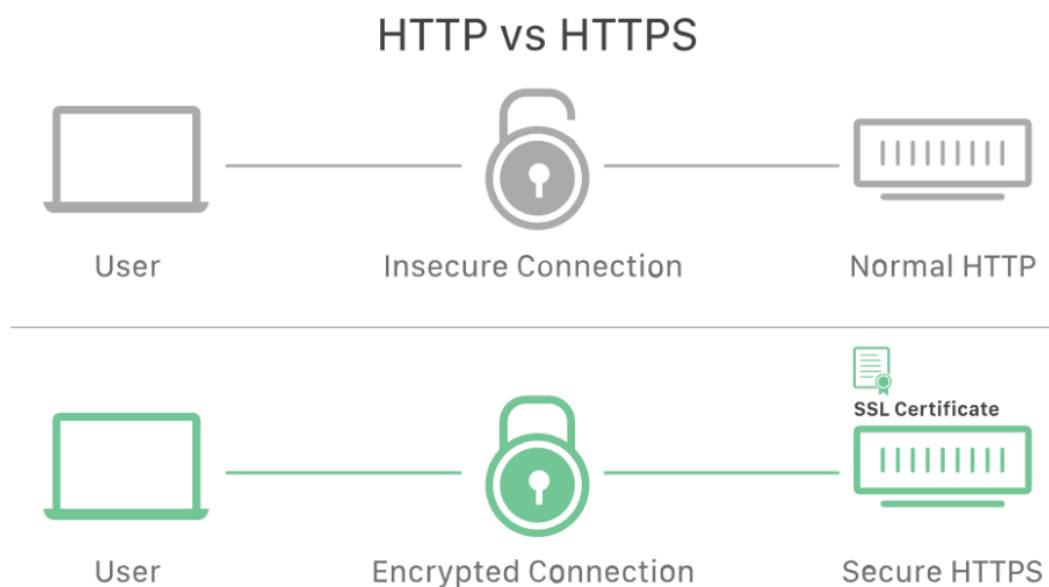
SSL/TLS là giao thức bảo mật nền tảng, được triển khai rộng rãi trong các hệ thống công nghệ thông tin hiện đại nhằm bảo vệ dữ liệu truyền qua mạng. Nhờ khả năng mã hóa, đảm bảo toàn vẹn và xác thực danh tính các thực thể tham gia giao tiếp, SSL/TLS trở thành thành phần không thể thiếu trong các dịch vụ Internet, hệ thống doanh nghiệp và hạ tầng số.

Các ứng dụng của SSL/TLS trải rộng từ dịch vụ web công cộng, hệ thống thư điện tử, mạng riêng ảo cho đến các kiến trúc hiện đại như điện toán đám mây và microservices.

### 5.2 Ứng dụng trong bảo mật Web (HTTPS)

Ứng dụng phổ biến và dễ nhận biết nhất của SSL/TLS là HTTPS (HTTP Secure).

- SSL/TLS mã hóa dữ liệu trao đổi giữa trình duyệt và máy chủ web.
- Ngăn chặn các cuộc tấn công nghe lén (sniffing) và tấn công trung gian (Man-in-the-Middle).
- Xác thực danh tính website thông qua chứng chỉ số, giúp người dùng nhận biết website hợp pháp.



Hình 10 HTTP và HTTPS

HTTPS đặc biệt quan trọng đối với:

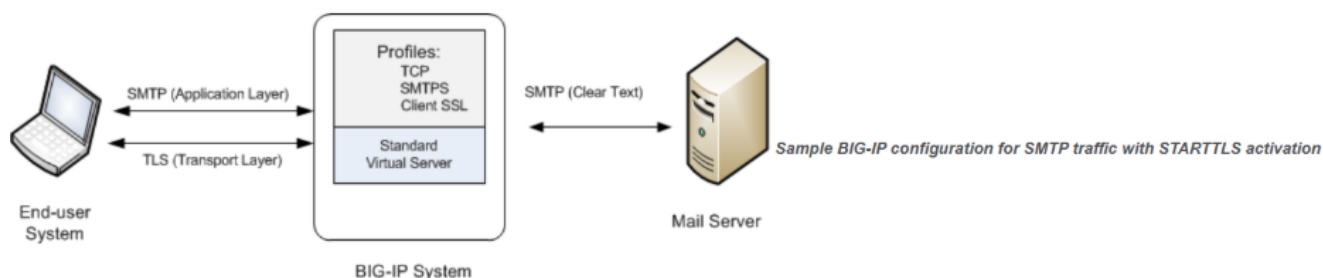
- Website thương mại điện tử
- Ngân hàng điện tử
- Cổng thanh toán trực tuyến
- Hệ thống quản lý thông tin cá nhân

Việc triển khai HTTPS hiện nay được coi là yêu cầu bắt buộc đối với các website cung cấp dịch vụ trên Internet.

### 5.3 Ứng dụng trong bảo mật thư điện tử

SSL/TLS được sử dụng để bảo vệ các giao thức thư điện tử phổ biến như:

- SMTPS (SMTP over TLS): bảo mật quá trình gửi email
- POP3S và IMAPS: bảo mật quá trình nhận email



Hình 11 Sơ đồ cấu hình SMTP với STARTTLS qua BIG-IP

Vai trò của SSL/TLS trong thư điện tử:

- Mã hóa nội dung email khi truyền trên mạng
- Bảo vệ thông tin tài khoản (username, password)
- Hạn chế nguy cơ bị đánh cắp thông tin hoặc giả mạo máy chủ mail

Điều này đặc biệt quan trọng trong môi trường doanh nghiệp, nơi email thường chứa dữ liệu nhạy cảm.

### 5.4 Ứng dụng trong VPN và truy cập từ xa

SSL/TLS được sử dụng làm nền tảng cho SSL VPN, cho phép người dùng truy cập hệ thống nội bộ thông qua Internet một cách an toàn.

Đặc điểm của SSL VPN:

- Không cần cấu hình phức tạp ở phía người dùng
- Hoạt động tốt qua tường lửa và NAT
- Mã hóa toàn bộ dữ liệu truyền giữa client và hệ thống nội bộ

SSL/TLS trong VPN giúp:

- Bảo vệ dữ liệu truy cập từ xa
- Xác thực người dùng
- Đảm bảo an toàn cho nhân sự làm việc từ xa

### 5.5 Ứng dụng trong các dịch vụ thư mục và xác thực

Trong các hệ thống quản lý tập trung, SSL/TLS được sử dụng để bảo mật các giao thức xác thực và thư mục, tiêu biểu là LDAPS (LDAP over SSL/TLS)

Vai trò:

- Mã hóa thông tin xác thực người dùng
- Bảo vệ dữ liệu tài khoản, mật khẩu
- Ngăn chặn việc thu thập thông tin đăng nhập trên mạng nội bộ

LDAPS thường được triển khai trong:

- Active Directory
- Hệ thống quản lý người dùng doanh nghiệp
- Các dịch vụ xác thực tập trung

## 5.6 Ứng dụng trong API và kiến trúc Microservices

Trong các hệ thống hiện đại sử dụng API và microservices, SSL/TLS đóng vai trò then chốt trong việc bảo vệ giao tiếp giữa các dịch vụ.

- Các API thường được triển khai trên nền HTTPS
- SSL/TLS bảo vệ dữ liệu trao đổi giữa các dịch vụ
- Hỗ trợ Mutual TLS (mTLS) để xác thực hai chiều giữa client và server

Ứng dụng này đặc biệt quan trọng trong:

- Kiến trúc Zero Trust
- Hệ thống phân tán
- Nền tảng cloud-native

## 5.7 Ứng dụng trong điện toán đám mây và container

SSL/TLS được tích hợp sâu trong các nền tảng điện toán đám mây và container:

- Bảo mật kết nối giữa người dùng và dịch vụ cloud
- Bảo vệ giao tiếp giữa các container
- Mã hóa dữ liệu khi truyền giữa các vùng (region, zone)

Các nền tảng như AWS, Azure, Google Cloud đều yêu cầu hoặc mặc định sử dụng SSL/TLS cho hầu hết các dịch vụ.

## 5.8 Ứng dụng trong bảo mật truyền file

SSL/TLS được sử dụng để bảo mật các giao thức truyền file FTPS (FTP over SSL/TLS)

Vai trò:

- Mã hóa dữ liệu và thông tin đăng nhập
- Bảo vệ file khỏi bị đánh cắp hoặc chỉnh sửa
- Thay thế FTP truyền thống vốn không có cơ chế bảo mật

## CHƯƠNG 6. THỰC NGHIỆM TRIỂN KHAI SSL/TLS

Trong chương này, nhóm tiến hành xây dựng một môi trường thử nghiệm cục bộ, cài đặt và cấu hình một máy chủ web Nginx hoạt động trên giao thức HTTPS sử dụng chứng chỉ SSL tự ký (self-signed certificate)

### 6.1 Các bước cài đặt

- Thiết lập một trang web thử nghiệm chạy trên nginx. Do trang web chưa được cấu hình SSL, nên giao thức hoạt động vẫn là HTTP:



### Trang web Thu nhien

- Sử dụng OpenSSL tự tạo một chứng chỉ SSL tự ký:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/nginx/ssl/demoapp.key -out /etc/nginx/ssl/demoapp.crt
```



- -x509: Yêu cầu xuất ra một chứng chỉ tự cấp thay vì một yêu cầu ký chứng chỉ (CSR).
- -nodes: Bỏ qua việc đặt mật khẩu bảo vệ cho khóa riêng, giúp Nginx có thể tự động đọc file khi khởi động lại máy chủ.
- -days 365: Đặt thời hạn hiệu lực của chứng chỉ là 365 ngày.
- -newkey rsa:2048: Tạo đồng thời một khóa RSA mới với độ dài 2048 bit.
- -keyout và -out: Chỉ định đường dẫn lưu trữ khóa riêng (.key) và chứng chỉ (.crt).
- Sau khi tạo khóa, hệ thống yêu cầu nhập thông tin cho chứng chỉ:



```

You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:demoapp.local
Email Address []:
student@labtainer-VirtualBox:~$

```

- Sau khi đã tạo thành công và sở hữu các tệp tin lưu trữ chứng chỉ, bước tiếp theo là khai báo và ràng buộc các vật liệu này vào cấu trúc định tuyến của một hệ thống Nginx Web Server. Kiến trúc an toàn mạng thường phân chia khối lệnh Server (server block) thành hai cơ chế độc lập: khối lắng nghe cổng HTTP mặc định để thực hiện chuyển hướng bắt buộc, và khối lắng nghe SSL/TLS để thực thi việc giải mã lưu lượng:

```

server {
    listen 80;
    server_name demoapp.local;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name demoapp.local;

    ssl_certificate /etc/ssl/certs/demoapp.crt;
    ssl_certificate_key /etc/ssl/private/demoapp.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}

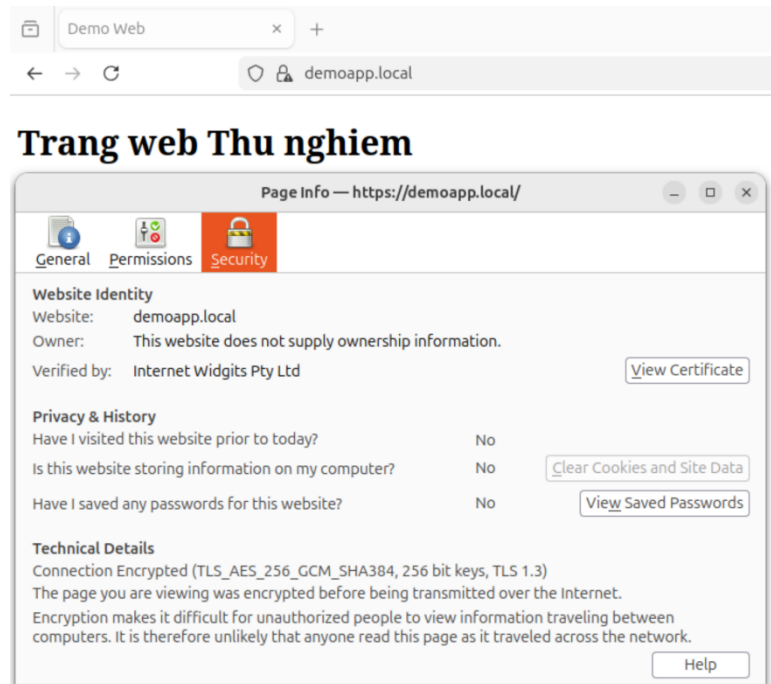
```

- `return 301 https://$host$request_uri;`: Đây là lệnh chuyển hướng. Khi người dùng cố gắng truy cập trang web bằng `http://`, máy chủ sẽ lập tức chuyển hướng vĩnh viễn (mã trạng thái 301) sang giao thức `https://` tương ứng.
- `listen 443 ssl;`: Chỉ định máy chủ lắng nghe kết nối trên cổng 443 và kích hoạt mô-đun SSL/TLS để mã hóa dữ liệu.
- `server_name demoapp.local;`: Xác định tên miền áp dụng.
- `ssl_certificate` và `ssl_certificate_key`: Cung cấp đường dẫn tuyệt đối trỏ tới file chứng chỉ (`demoapp.crt`) và file khóa riêng tư (`demoapp.key`) đã được tạo trước đó trên máy chủ (lưu trong `/etc/ssl/`).
- `ssl_protocols TLSv1.2 TLSv1.3;`: Đây là cấu hình bảo mật nâng cao nhằm chỉ cho phép các phiên bản giao thức an toàn là TLS 1.2 và TLS 1.3 hoạt động, đồng thời từ chối các phiên bản SSL/TLS cũ đã có lỗ hổng.

- ssl\_ciphers HIGH:!aNULL:!MD5;; Quy định các bộ thuật toán mã hóa (Cipher Suites) được phép dùng. Tùy chọn này yêu cầu sử dụng mã hóa mức độ cao (HIGH) và loại trừ hoàn toàn các thuật toán yếu, không xác thực (!aNULL) hoặc đã bị phá vỡ như MD5 (!MD5).

## 6.2 Kết quả thu được

- Mở hộp thoại thông tin bảo mật của trình duyệt:



- Trang web được xác thực bởi Internet Widgits Pty Ltd (tên tổ chức mặc định mà OpenSSL tự động điền vào)
- Trình duyệt xác nhận kết nối đã được mã hóa (Connection Encrypted).
- Giao thức bảo mật được sử dụng là TLS 1.3 (phiên bản hiện đại và an toàn nhất hiện nay).
- Bộ thuật toán mã hóa (Cipher Suite) được thỏa thuận là TLS\_AES\_256\_GCM\_SHA384 với độ dài khóa 256 bit. Đây là một bộ mã hóa cực kỳ mạnh mẽ, đảm bảo tính bí mật và toàn vẹn của dữ liệu truyền đi.

=> Trang web đã được chạy trên nền giao thức HTTPS. Dữ liệu được truyền đi đã đảm bảo tính bảo mật.

- Kiểm tra chứng chỉ SSL của trang web:

| Certificate     |                                                                            |  |
|-----------------|----------------------------------------------------------------------------|--|
| demoapp.local   |                                                                            |  |
| Subject Name    |                                                                            |  |
| Country         | AU                                                                         |  |
| State/Province  | Some-State                                                                 |  |
| Organization    | Internet Widgits Pty Ltd                                                   |  |
| Common Name     | demoapp.local                                                              |  |
| Issuer Name     |                                                                            |  |
| Country         | AU                                                                         |  |
| State/Province  | Some-State                                                                 |  |
| Organization    | Internet Widgits Pty Ltd                                                   |  |
| Common Name     | demoapp.local                                                              |  |
| Validity        |                                                                            |  |
| Not Before      | Mon, 16 Feb 2026 01:21:45 GMT                                              |  |
| Not After       | Tue, 16 Feb 2027 01:21:45 GMT                                              |  |
| Public Key Info |                                                                            |  |
| Algorithm       | RSA                                                                        |  |
| Key Size        | 2048                                                                       |  |
| Exponent        | 65537                                                                      |  |
| Modulus         | C6:02:2A:B9:CD:84:99:50:9B:D4:7B:03:D9:85:FD:B2:95:17:30:16:CC:28:07:34... |  |

- Chứng chỉ tự ký: Thông tin Chủ thể (Subject) và Tổ chức cấp (Issuer) hoàn toàn giống nhau, sử dụng các dữ liệu mặc định của OpenSSL.
- Tên miền (Common Name): là demoapp.local, khớp với cấu hình máy chủ.
- Thời hạn (Validity): Có hiệu lực đúng 1 năm (365 ngày), tương ứng với việc dùng cờ -days 365 lúc tạo.
- Khóa công khai (Public Key): Dùng chuẩn RSA 2048-bit, khớp với tham số lệnh -newkey rsa:2048.

## KẾT LUẬN

Trong khuôn khổ báo cáo, nhóm đã thực hiện nghiên cứu tổng quan và chuyên sâu về giao thức bảo mật SSL/TLS – nền tảng bảo mật quan trọng nhất trong truyền thông Internet hiện nay. Báo cáo đã trình bày đầy đủ các nội dung từ lịch sử phát triển, vị trí trong mô hình mạng, kiến trúc giao thức, cơ chế hoạt động của TLS 1.2, quá trình bắt tay, dẫn xuất khóa, cơ chế khôi phục phiên, cho đến các ứng dụng thực tiễn trong HTTPS, thư điện tử, VPN và các hệ thống hiện đại.

Bên cạnh đó, nhóm cũng phân tích các lỗ hổng và dạng tấn công phổ biến như Downgrade Attack, POODLE, BEAST, SWEET32, Heartbleed, MITM và SSL Stripping, đồng thời đề xuất các biện pháp phòng chống như vô hiệu hóa giao thức cũ, sử dụng cipher suite mạnh, bật HSTS, cập nhật thư viện và triển khai TLS 1.3. Việc thực nghiệm cài đặt và cấu hình một website chạy HTTPS bằng chứng chỉ SSL tự ký trên Nginx đã giúp minh chứng rõ ràng cơ chế hoạt động và quy trình triển khai SSL/TLS trong thực tế.

Qua quá trình thực hiện đề tài, có thể khẳng định rằng việc cấu hình đúng và cập nhật thường xuyên giao thức TLS đóng vai trò quyết định trong việc đảm bảo an toàn thông tin cho hệ thống. Trong tương lai, hướng nghiên cứu có thể mở rộng sang phân tích sâu hơn về TLS 1.3, cơ chế 0-RTT, Mutual TLS (mTLS), cũng như các kỹ thuật tấn công và phòng thủ trong môi trường thực tế phức tạp hơn.

Báo cáo góp phần củng cố kiến thức lý thuyết và kỹ năng thực hành về bảo mật truyền thông mạng, đồng thời tạo nền tảng cho các nghiên cứu và triển khai chuyên sâu hơn trong lĩnh vực an toàn thông tin.

## TÀI LIỆU THAM KHẢO

- [1] T. Dierks and E. Rescorla, The Transport Layer Security (TLS) Protocol Version 1.2, RFC 5246, IETF, 2008.
- [2] E. Rescorla, The Transport Layer Security (TLS) Protocol Version 1.3, RFC 8446, IETF, 2018.
- [3] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th Edition, Pearson, 2017.
- [4] A. Freier, P. Karlton, and P. Kocher, The SSL Protocol Version 3.0, Netscape Communications, 1996.
- [5] OpenSSL Project, OpenSSL Documentation, <https://www.openssl.org>.
- [6] Nginx, Nginx SSL/TLS Configuration Guide, <https://nginx.org>